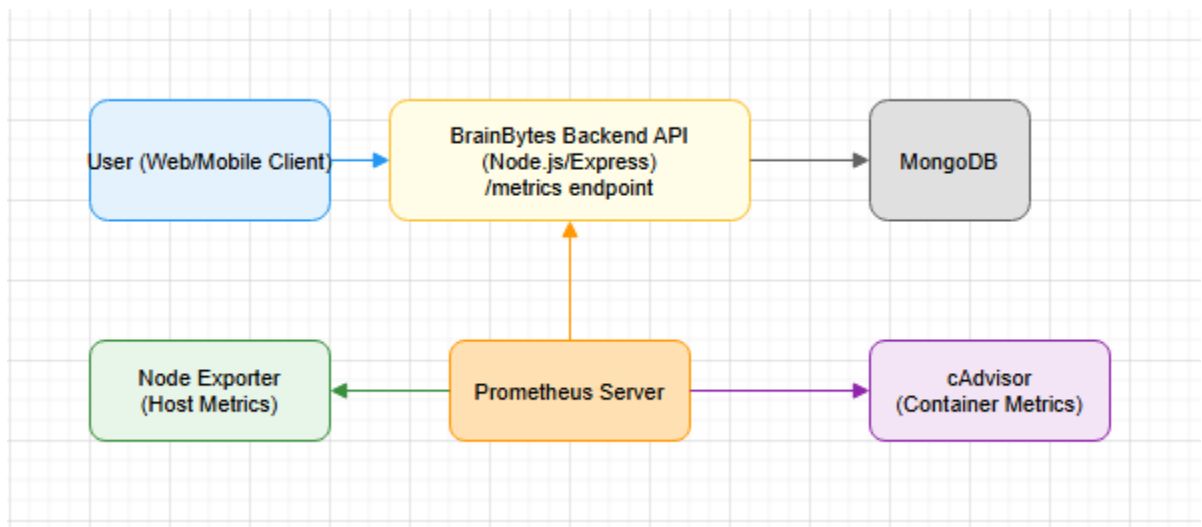


BrainBytes Monitoring System Documentation

1. System Architecture Overview

Architecture Diagram



The diagram depicts the metrics monitoring architecture for the BrainBytes application. It illustrates the interaction between application components, metric collection tools, and monitoring services.

Components to include:

BrainBytes Backend ([Node.js/Express](#)):

Handles user requests, business logic, and exposes application metrics via /metrics using prom-client.

MongoDB:

Stores application data (users, messages, etc.).

Prometheus:

Scrapes metrics from the backend and other exporters, stores them in a time-series database, and evaluates alerting/recording rules.

Node Exporter:

Collects host-level metrics (CPU, memory, disk, etc.) from the server running the containers.

cAdvisor:

Collects container-level metrics (CPU, memory, network, etc.) for each Docker container.

Alertmanager:

Receives alerts from Prometheus and routes notifications to channels like email or Slack.

Docker Containers:

All services (backend, database, exporters) run as isolated containers.

Traffic Simulator:

Generates synthetic load to test the system and metrics.

Data Flow Description:

1. Metrics Collection:

- a. The backend uses prom-client to instrument and collect custom metrics (counters, gauges, histograms).
- b. Host and container metrics are collected by Node Exporter and cAdvisor, respectively.

2. Metrics Exposure:

- a. The backend exposes metrics at /metrics.
- b. Node Exporter and cAdvisor expose their own endpoints.

3. Scraping & Storage:

- a. Prometheus scrapes all /metrics endpoints at defined intervals.
- b. All metrics are stored in Prometheus's time-series database.

4. Recording Rules:

- a. Prometheus uses recording rules to precompute and store frequently queried expressions for efficiency.

5. Alerting:

- a. Prometheus evaluates alerting rules.
- b. When thresholds are breached, alerts are sent to Alertmanager.

6. Alert Routing:

- a. Alertmanager routes alerts to notification channels (email, Slack, etc.).

2. Metrics Catalog

Metric Name	Type	Description	Labels	Example Query	Domain
brainbytes_messages_per_subject_total	Counter	Total number of messages per subject	subject	sum by (subject) (brainbytes_messages_per_subject_total)	Business
brainbytes_mobile_response_time_seconds	Histogram	Histogram of response times for mobile clients in seconds	None	histogram_quantile(0.95, sum(rate(brainbytes_mobile_response_time_seconds_bucket[5m])) by (le))	Application
brainbytes_data_usage_bytes_total	Counter	Total data usage in bytes (sent/received)	direction, clientType	sum by (direction, clientType) (brainbytes_data_usage_bytes_total)	System
brainbytes_intermittent_connectivity_events	Gauge	Current number of detected intermittent connectivity	None	brainbytes_intermittent_connectivity_events	Application

		events (mobile)			
brainbytes_ai_response_time_seconds	Histogram	Histogram of AI response times in seconds	None	histogram_quantile(0.95, sum(rate(brainbytes_ai_response_time_seconds_bucket[5m])) by (le))	Application
brainbytes_daily_active_users	Gauge	Number of unique user emails who sent a message in the last 24 hours	None	brainbytes_daily_active_users	Business

3. Query Reference Guide:

PromQL Query Reference

1. Total HTTP Requests (per endpoint)

Query:

```
sum by (endpoint) (rate(brainbytes_http_requests_total[5m]))
```

Description: Shows the per-second request rate for each API endpoint.

Interpretation: High rates on certain endpoints may indicate popular features or potential abuse.

2. Average HTTP Request Duration (per endpoint)

Query:

```
sum by (endpoint)
(rate(brainbytes_http_request_duration_seconds_sum[5m])) / sum
by (endpoint)
(rate(brainbytes_http_request_duration_seconds_count[5m]))
```

Description: Shows the average response time for each endpoint.

Interpretation: Spikes may indicate backend slowness or issues with specific endpoints.

3. Active Tutoring Sessions

Query:

brainbytes_active_sessions

Description: Shows the current number of active tutoring sessions.

Interpretation: A sudden drop may indicate user disengagement or system issues.

4. Messages Per Subject (rate)

Query:

sum by (subject)

(rate(brainbytes_messages_per_subject_total[5m]))

Description: Shows the rate of messages for each subject.

Interpretation: Helps identify which subjects are most active or trending.

5. AI Response Time (average)

Query:

rate(brainbytes_ai_response_time_seconds_sum[5m]) /

rate(brainbytes_ai_response_time_seconds_count[5m])

Description: Shows the average AI response time.

Interpretation: Higher values may indicate AI service delays.

6. Mobile Response Time (average)

Query:

rate(brainbytes_mobile_response_time_seconds_sum[5m]) /

rate(brainbytes_mobile_response_time_seconds_count[5m])

Description: Shows the average response time for mobile clients.

Interpretation: Useful for monitoring mobile user experience; spikes may indicate network or server issues.

7. Data Usage by Client Type (sent)

Query:

sum by (clientType)

(increase(brainbytes_data_usage_bytes_total{direction="sent"}[1h]))

Description: Shows bytes sent by client type over the last hour.

Interpretation: High values may indicate heavy downloads or large responses.

8. Data Usage by Client Type (received)

Query:

sum by (clientType)

(increase(brainbytes_data_usage_bytes_total{direction="received"}[1h]))

Description: Shows bytes received by client type over the last hour.

Interpretation: High values may indicate large uploads or frequent requests.

9. Intermittent Connectivity Events (rate)

Query:

`rate(brainbytes_intermittent_connectivity_events[5m])`

Description: Shows the rate of detected intermittent connectivity events.

Interpretation: Spikes may indicate network instability for mobile users.

10. Daily Active Users

Query:

`brainbytes_daily_active_users`

Description: Shows the number of unique users active in the last 24 hours.

Interpretation: Useful for tracking user engagement and growth trends.

4. Alert Rules Documentation:

1. BackendDown

Rule: `up{job="brainbytes-backend"} == 0` for 1m

Threshold: Backend is down for 1 minute.

Justification: Indicates the backend is unreachable or crashed.

Response:

- Check backend server/container status
- Review logs for crashes or resource exhaustion
- Restart service if needed

Routing: Severity: critical. Routed to on-call DevOps/SRE.

2. HighErrorRate

Rule: `increase(brainbytes_http_requests_total{status=~"5.."}[5m]) > 5` for 5m

Threshold: More than 5 HTTP 5xx errors in 5 minutes.

Justification: Indicates backend/server errors affecting users.

Response:

- Check server logs for stack traces or error messages
- Identify failing endpoints and recent deployments
- Roll back or hotfix as needed

Routing: Severity: warning. Routed to backend and DevOps.

3. **HighRequestLatency**

Rule: `histogram_quantile(0.95, sum(rate(brainbytes_http_request_duration_seconds_bucket[5m])) by (le)) > 1` for 5m

Threshold: 95th percentile request latency > 1 second for 5 minutes.

Justification: Slow responses degrade user experience.

Response:

- Check for database or external API slowness
- Review recent code changes
- Scale up resources if needed

Routing: Severity: warning. Routed to backend and SRE.

4. **NodeExporterDown**

Rule: `up{job="node-exporter"} == 0` for 1m

Threshold: Node Exporter is down for 1 minute.

Justification: System metrics are unavailable; may indicate node issues.

Response:

- Check node-exporter process/container
- Restart if needed

Routing: Severity: critical. Routed to DevOps/SRE.

5. **CadvisorDown**

Rule: `up{job="cadvisor"} == 0` for 1m

Threshold: cAdvisor is down for 1 minute.

Justification: Container metrics are unavailable; may indicate node/container issues.

Response:

- Check cAdvisor process/container
- Restart if needed

Routing: Severity: critical. Routed to DevOps/SRE.

6. **HighAIResponseTime**

Rule: `rate(brainbytes_ai_response_time_seconds_sum[5m]) / rate(brainbytes_ai_response_time_seconds_count[5m]) > 5` for 5m

Threshold: Average AI response time > 5 seconds for 5 minutes.

Justification: Indicates AI service or integration issues.

Response:

- Check AI service health and logs

- Restart or scale AI service if needed
- Notify users if degraded

Routing: Severity: warning. Routed to AI/ML and backend teams.

7. HighMobileResponseTime

Rule: $\text{rate}(\text{brainbytes_mobile_response_time_seconds_sum}[5\text{m}]) / \text{rate}(\text{brainbytes_mobile_response_time_seconds_count}[5\text{m}]) > 5$ for 5m

Threshold: Average mobile response time > 5 seconds for 5 minutes.

Justification: Mobile users are experiencing slowness.

Response:

- Check network and server logs for mobile requests
- Investigate CDN or mobile-specific issues

Routing: Severity: warning. Routed to SRE and mobile app team.

8. HighDataUsageSent

Rule:

$\text{sum}(\text{increase}(\text{brainbytes_data_usage_bytes_total}\{\text{direction}=\text{"sent"}\}[1\text{h}])) > 1\text{e}9$ for 10m

Threshold: More than 1GB sent in the last hour.

Justification: May indicate abuse, DDoS, or large downloads.

Response:

- Check for abnormal traffic patterns
- Block abusive IPs if needed

Routing: Severity: warning. Routed to Security and DevOps.

9. HighDataUsageReceived

Rule:

$\text{sum}(\text{increase}(\text{brainbytes_data_usage_bytes_total}\{\text{direction}=\text{"received"}\}[1\text{h}])) > 1\text{e}9$ for 10m

Threshold: More than 1GB received in the last hour.

Justification: May indicate large uploads or abuse.

Response:

- Review upload endpoints and logs
- Enforce rate limits if needed

Routing: Severity: warning. Routed to Security and backend.

10. FrequentIntermittentConnectivityEvents

Rule: $\text{increase}(\text{brainbytes_intermittent_connectivity_events}[10\text{m}]) > 10$ for 10m

Threshold: More than 10 events in 10 minutes.

Justification: Indicates network instability for mobile users.

Response:

- Check mobile network health and server logs
- Notify users if widespread

Routing: Severity: warning. Routed to SRE and mobile team.

11. LowDailyActiveUsers

Rule: `brainbytes_daily_active_users < 5` for 1h

Threshold: Less than 5 daily active users in the last 24 hours.

Justification: May indicate system issues or user churn.

Response:

- Check for outages or login issues
- Notify product and marketing teams

Routing: Severity: info. Routed to product owner and DevOps.

12. NoMessagesForSubject

Rule: `sum(rate(brainbytes_messages_per_subject_total[30m])) by (subject) == 0` for 30m

Threshold: No messages for a subject in 30 minutes.

Justification: May indicate a bug or loss of engagement in a subject.

Response:

- Check frontend and backend logs for errors
- Investigate recent changes to subject handling

Routing: Severity: info. Routed to backend and product team.

Alert Grouping and Routing

- Grouping: Alerts are grouped by severity (critical, warning, info) and by affected service (API, AI, mobile, etc.).

- Routing:

- Critical alerts (e.g., backend down, node/cadvisor down) go to on-call DevOps/SRE via PagerDuty/Slack/email.

- Warning alerts (performance, usage, mobile, AI) go to backend, AI/ML, SRE, or mobile teams as appropriate.

- Info/product/engagement alerts go to product owner and marketing.

- Escalation: If an alert is not acknowledged within 10 minutes, it is escalated to the next level of support.

Traffic Simulation Documentation

1. simulate-traffic-advanced.js

Purpose:

Simulates realistic user behavior by generating continuous POST and GET requests to the backend. The simulation includes valid and erroneous requests from both desktop and mobile clients, and dynamically adjusts traffic patterns to mimic peak and off-peak hours.

Setup Steps:

- Install required dependencies (if not already installed).

```
npm install node-fetch
```

- Start the backend server at `http://localhost:3000`.

- Run the simulation script.

```
node simulate-traffic-advanced.js
```

Simulation Behavior:

- Sends requests using randomized user identities and subject fields.
- Includes a mix of valid and intentionally erroneous API calls.
- Adjusts the rate of requests based on simulated time-of-day cycles (e.g., high traffic during peak hours, low traffic during quiet periods).
- Logs every request with:
 - HTTP status
 - Client type (mobile or desktop)
 - Estimated data usage

To Stop the Simulation:

Press Ctrl + C in the terminal.

2. simulate-scenarios.js

Purpose:

Executes predefined traffic scenarios to test system performance, error handling, and resource management. Useful for validating Prometheus metrics and alert rule responses.

Setup Steps:

- Install required dependencies (if not already installed).

```
npm install node-fetch
```

- Ensure the backend server is running at `http://localhost:3000`.

- Execute the script with the desired scenario mode.

Available Scenarios:

- High Load:

```
node simulate-scenarios.js highload
```

- Sends 100 valid requests at a rate of 10 requests per second.

- Error Spike:

```
node simulate-scenarios.js errors
```

- Sends 50 invalid requests designed to trigger error handling mechanisms.

- Resource Constraint (Slow):

```
node simulate-scenarios.js slow
```

- Sends 20 valid requests at 5-second intervals, simulating limited processing resources.

Interpretation & Use:

Each scenario is designed to trigger specific Prometheus metrics and alert rules.

- Monitor the Prometheus dashboards for expected spikes or anomalies.
- Confirm that configured alerting mechanisms fire appropriately during each test.
- Use results to validate and tune detection thresholds and response actions.

Localization Documentation

1. Mobile-First and Intermittent Connectivity

To account for network realities across the Philippines, the system includes custom metrics:

- **brainbytes_mobile_response_time_seconds**: Tracks response latency experienced by mobile clients.
- **brainbytes_intermittent_connectivity_events**: Detects and quantifies unstable mobile network conditions.

These metrics help identify slowdowns and intermittent connectivity, which are common due to variable infrastructure quality throughout the region.

2. Data Usage Awareness

The metric **brainbytes_data_usage_bytes_total**, labeled by **clientType** (mobile or desktop), monitors:

- Bytes sent
- Bytes received

This supports optimization for users with limited or costly data plans—an especially relevant consideration in the local setting.

3. Error and Latency Tolerance

Alert thresholds are tuned to the realities of regional connectivity:

- **Latency**: Alerts for mobile response times trigger only when sustained above 5 seconds for 5 minutes—avoiding false alarms from brief spikes.
- **Connectivity**: Intermittent connectivity alerts are triggered only if more than 10 events occur within a 10-minute window—filtering out transient drops.

Threshold Adaptations for Local Connectivity

- **Mobile Response Time**:
Triggered only if the average exceeds 5 seconds for 5 consecutive minutes, accommodating common mobile fluctuations.

- **Intermittent Connectivity:**
Alerts fire when more than 10 unstable events are detected in 10 minutes, surfacing persistent issues without reacting to brief outages.
- **Data Usage:**
Alerts are set at 1GB/hour to avoid false positives during regular usage while detecting misuse or inefficiencies that affect low-bandwidth users.

Cost Optimization for Cloud Resources

1. Efficient Metrics Collection

- Prometheus scrapes occur at balanced intervals to minimize load.
- Only critical metrics and label sets are collected, limiting cardinality and storage demand.

2. Alerting and Scaling Efficiency

- Alert rules are tuned to avoid reaction to transient anomalies.
- Scaling is adaptive and condition-specific (e.g., only scaling for sustained mobile latency or request rate increases).

3. Data Retention and Storage Management

- High-resolution metrics are retained short-term.
- Older data is downsampled to reduce long-term storage requirements.
- Prometheus recording rules simplify frequent queries, decreasing compute load.

4. Cloud Resource Monitoring

- Node Exporter and cAdvisor track host and container usage in real time.
- Insights from these tools help right-size VM/container allocations and prevent wasteful over-provisioning.

5. Traffic Simulation for Pre-Production Testing

- Scenario-based traffic scripts simulate various loads and user behaviors.
- Enables performance testing and tuning of resource thresholds before production scaling.
- Avoids unnecessary cloud spending through predictive profiling.

SCREENSHOT



```
[2025-07-21T02:49:03.080Z] POST /api/messages: 201 | clientType=mobile | reqBytes=169 | resBytes=22
[2025-07-21T02:49:04.462Z] POST /api/messages: 201 | clientType=mobile | reqBytes=171 | resBytes=22
[2025-07-21T02:49:05.576Z] GET /api/messages: 200 | clientType=desktop | reqBytes=61 | resBytes=22
[2025-07-21T02:49:06.075Z] POST /api/messages: 400 | clientType=desktop | reqBytes=115 | resBytes=22
[2025-07-21T02:49:08.816Z] POST /api/messages: 201 | clientType=mobile | reqBytes=171 | resBytes=22
[2025-07-21T02:49:11.476Z] POST /api/messages: 201 | clientType=desktop | reqBytes=168 | resBytes=22
[2025-07-21T02:49:14.437Z] POST /api/messages: 201 | clientType=mobile | reqBytes=169 | resBytes=22
[2025-07-21T02:49:16.638Z] POST /api/messages: 400 | clientType=mobile | reqBytes=114 | resBytes=22
[2025-07-21T02:49:18.998Z] POST /api/messages: 201 | clientType=mobile | reqBytes=171 | resBytes=22
[2025-07-21T02:49:20.544Z] POST /api/messages: 400 | clientType=desktop | reqBytes=115 | resBytes=22
[2025-07-21T02:49:23.196Z] GET /api/messages: 200 | clientType=desktop | reqBytes=61 | resBytes=22
[2025-07-21T02:49:25.144Z] POST /api/messages: 400 | clientType=mobile | reqBytes=114 | resBytes=22
[2025-07-21T02:49:27.374Z] POST /api/messages: 201 | clientType=mobile | reqBytes=165 | resBytes=22
[2025-07-21T02:49:29.341Z] GET /api/messages: 200 | clientType=mobile | reqBytes=60 | resBytes=22
[2025-07-21T02:49:30.764Z] POST /api/messages: 400 | clientType=mobile | reqBytes=114 | resBytes=22
[2025-07-21T02:49:32.327Z] GET /api/users: 200 | clientType=mobile | reqBytes=60 | resBytes=22
[2025-07-21T02:49:33.662Z] GET /api/messages: 200 | clientType=mobile | reqBytes=60 | resBytes=22
[2025-07-21T02:49:35.218Z] POST /api/messages: 400 | clientType=mobile | reqBytes=114 | resBytes=22
[2025-07-21T02:49:37.461Z] POST /api/messages: 400 | clientType=mobile | reqBytes=114 | resBytes=22
[2025-07-21T02:49:38.649Z] GET /api/messages: 200 | clientType=mobile | reqBytes=60 | resBytes=22
[2025-07-21T02:49:40.136Z] GET /api/users: 200 | clientType=desktop | reqBytes=61 | resBytes=22
[2025-07-21T02:49:43.752Z] GET /api/users: 200 | clientType=mobile | reqBytes=60 | resBytes=22
[2025-07-21T02:49:44.703Z] GET /api/messages: 200 | clientType=mobile | reqBytes=60 | resBytes=22
[2025-07-21T02:49:47.077Z] GET /api/users: 200 | clientType=desktop | reqBytes=61 | resBytes=22
[2025-07-21T02:49:49.450Z] GET /api/users: 200 | clientType=mobile | reqBytes=60 | resBytes=22
[2025-07-21T02:49:51.028Z] POST /api/messages: 201 | clientType=desktop | reqBytes=169 | resBytes=22
[2025-07-21T02:49:52.771Z] POST /api/messages: 201 | clientType=mobile | reqBytes=165 | resBytes=22
```

☒ Use local time ☐ Enable query history ☒ Enable autocomplete ☒ Enable highlighting ☒ Enable linter

Q brainbytes_active_sessions

Execute

Load time: 30ms Resolution: 1s Result series: 1

Table Graph

- 5m + < End time > Res. (s) Show Exemplars



brainbytes_active_sessions(environment="production", instance="brainbytes-backend", job="brainbytes-backend", service="brainbytes-backend")

Remove Panel

Q brainbytes_messages_per_subject_total

Execute

Load time: 4ms Resolution: 1s Result series: 5

Table Graph

- 5m + < End time > Res. (s) Show Exemplars



brainbytes_messages_per_subject_total(environment="production", instance="brainbytes-backend", job="brainbytes-backend", service="brainbytes-backend", subject="General")
brainbytes_messages_per_subject_total(environment="production", instance="brainbytes-backend", job="brainbytes-backend", service="brainbytes-backend", subject="History")
brainbytes_messages_per_subject_total(environment="production", instance="brainbytes-backend", job="brainbytes-backend", service="brainbytes-backend", subject="Language")
brainbytes_messages_per_subject_total(environment="production", instance="brainbytes-backend", job="brainbytes-backend", service="brainbytes-backend", subject="Math")
brainbytes_messages_per_subject_total(environment="production", instance="brainbytes-backend", job="brainbytes-backend", service="brainbytes-backend", subject="Technology")

Click: select series, CTRL + click: toggle multiple series

Remove Panel

Q brainbytes_http_requests_total

Execute

Load time: 3ms Resolution: 1s Result series: 4

Table Graph

- 5m + < End time > Res. (s) Show Exemplars



brainbytes_http_requests_total(api_endpoint="/api/messages", endpoint="/api/messages", environment="production", http_method="GET", instance="brainbytes-backend", job="brainbytes-backend", method="GET", service="brainbytes-backend", status="200")
brainbytes_http_requests_total(api_endpoint="/api/messages", endpoint="/api/messages", environment="production", http_method="POST", instance="brainbytes-backend", job="brainbytes-backend", method="POST", service="brainbytes-backend", status="200")